

EDGE DA 9
ANUSREE V
25MML0014

```
[28]: pip install opencv-python
```

Requirement already satisfied: opencv-python in /opt/anaconda3/envs/ml_lab/lib/python3.9/site-packages (4.12.0.88)
Requirement already satisfied: numpy<2.3.0, >=2 in /opt/anaconda3/envs/ml_lab/lib/python3.9/site-packages (from opencv-python) (2.0.2)
Note: you may need to restart the kernel to use updated packages.

```
[30]: import cv2
```

```
[32]: import cv2
```

```
cap = cv2.VideoCapture(0)  
  
print(cap.isOpened())  
  
cap.release()
```

```
True
```

```
[34]: import cv2
```

```
cap = cv2.VideoCapture(0)  
  
while True:  
    ret, frame = cap.read()  
  
    if not ret:  
        print("Failed to grab frame")  
        break  
  
    cv2.imshow("Laptop Camera", frame)  
  
    if cv2.waitKey(1) & 0xFF == ord('q'):  
        break  
  
cap.release()  
cv2.destroyAllWindows()
```

```
import cv2  
  
cap = cv2.VideoCapture(0)  
  
while True:  
    ret, frame = cap.read()  
  
    if not ret:  
        print("Failed to grab frame")  
        break  
  
    cv2.imshow("Laptop Camera", frame)  
  
    if cv2.waitKey(1) & 0xFF == ord('q'):  
        break  
  
cap.release()  
cv2.destroyAllWindows()
```

```
#Display video properties
```

```
import cv2  
  
cap = cv2.VideoCapture("sample.mp4")  
  
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))  
fps = cap.get(cv2.CAP_PROP_FPS)  
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))  
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))  
  
print("Total Frames:", total_frames)  
print("FPS:", fps)  
print("Width:", width)  
print("Height:", height)  
  
cap.release()
```

Total Frames: 2398
FPS: 59.94005994005994
Width: 1280
Height: 720

```
[48]: import cv2

# open video
cap = cv2.VideoCapture("sample.mp4")

# read first frame
ret, frame = cap.read()

if ret:

    # print pixel values of the frame
    print("Pixel values of original frame:")
    print(frame)

    # convert to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # print grayscale pixel values
    print("Pixel values of grayscale frame:")
    print(gray)

    # display images
    cv2.imshow("Original Frame", frame)
    cv2.imshow("Grayscale Frame", gray)

    cv2.waitKey(0)

cap.release()
cv2.destroyAllWindows()
```

```
Pixel values of original frame:
[[[ 54  68  60]
  [ 54  68  60]
  [ 54  68  60]
  ...
  [ 18  13   8]
  [ 18  13   8]
  [ 18  13   8]]]
```

```
[125 120 120]
[124 119 119]
...
[ 38  33  33]
[ 38  33  33]
[ 38  33  33]]]
```

```
[[127 123 123]
 [124 119 119]
 [118 113 113]
 ...
 [ 39  34  34]
 [ 39  34  34]
 [ 39  34  34]]]
```

```
[[116 111 111]
 [111 107 107]
 [103  98  98]
 ...
 [ 39  34  34]
 [ 39  34  34]
 [ 39  34  34]]]
```

```
Pixel values of grayscale frame:
[[ 64  64  64 ... 12 12 12]
 [ 64  64  64 ... 12 12 12]
 [ 62  62  62 ... 12 12 12]
 ...
 [122 121 120 ... 34 34 34]
 [123 120 114 ... 35 35 35]
 [112 107  99 ... 35 35 35]]]
```

```
[50]: import cv2
```



One image is original frame of the video and other one is gray scale image

```
import cv2

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()

    if not ret:
        break

    # draw rectangle
    cv2.rectangle(frame, (100,100), (300,300), (0,255,0), 3)

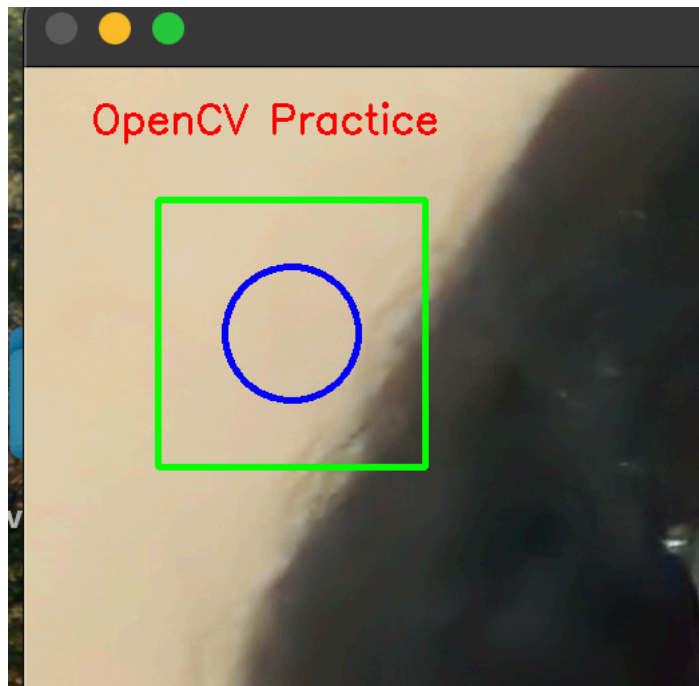
    # draw circle
    cv2.circle(frame, (200,200), 50, (255,0,0), 3)

    # add text
    cv2.putText(frame, "OpenCV Practice", (50,50),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)

    cv2.imshow("Shapes", frame)

    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```



#CANNY EDGE DETECTION

```
import cv2

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()

    if not ret:
        break

    edges = cv2.Canny(frame, 100, 200)

    cv2.imshow("Edges", edges)

    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

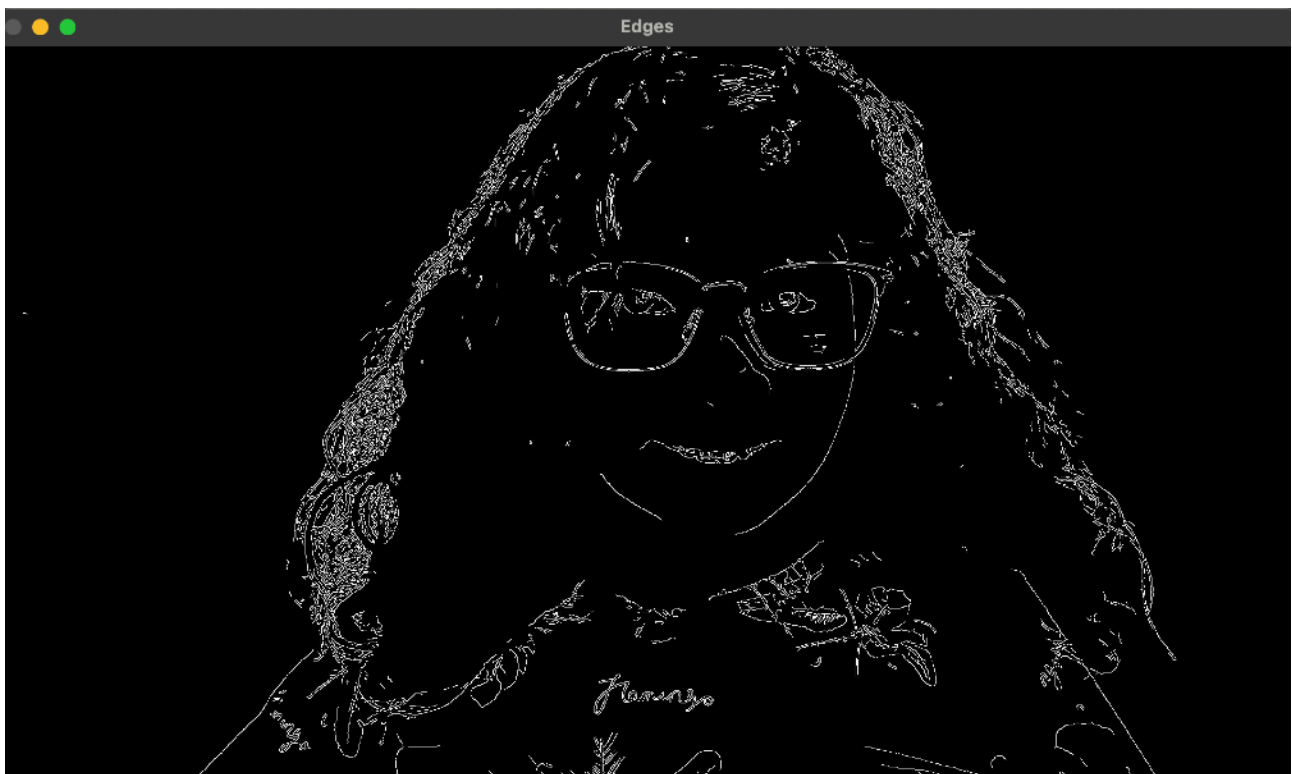


Image representation
Grayscale conversion
Gaussian smoothing
Gradient magnitude
Gradient direction
Double threshold

#NEGATIVE IMAGE

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)

while True:

    ret, frame = cap.read()
    if not ret:
        break

    # increase brightness and contrast
    alpha = 1.6
    beta = 30
    bright = cv2.convertScaleAbs(frame, alpha=alpha, beta=beta)

    # sharpening kernel
    kernel = np.array([[0,-1,0],
                       [-1,5,-1],
                       [0,-1,0]])

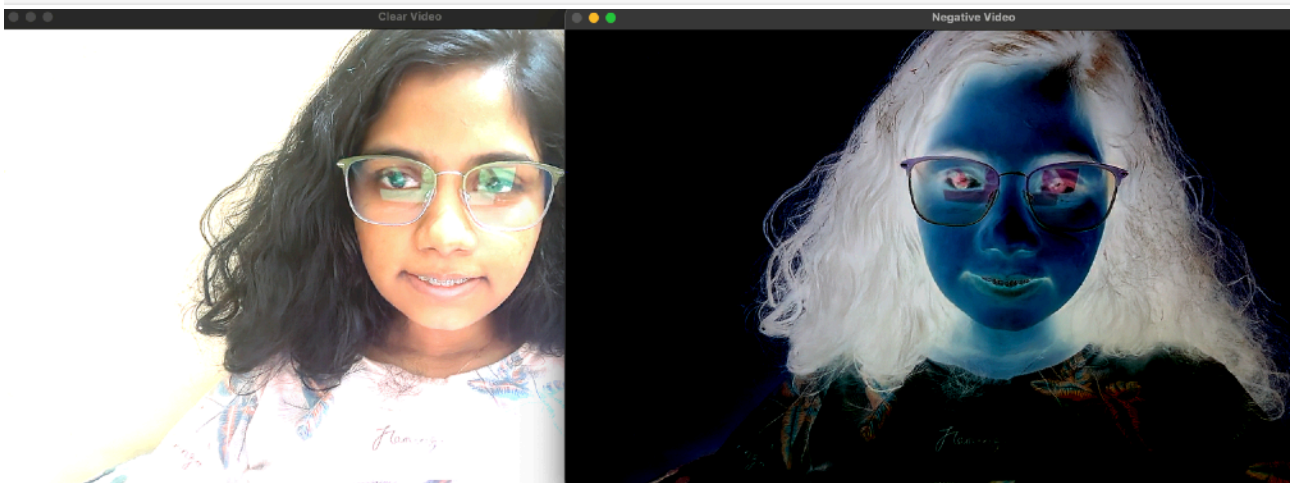
    sharp = cv2.filter2D(bright, -1, kernel)

    # negative image
    negative = 255 - sharp

    # display
    cv2.imshow("Clear Video", sharp)
    cv2.imshow("Negative Video", negative)

    # press q to quit
    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```



Operation	Formula
Image representation	$(I(x,y))$
Brightness & contrast	$(I' = \alpha I + \beta)$
Sharpening filter	$(I_s = I * K)$
Negative image	$(I_n = 255 - I)$
Video sequence	$(V = \{I_1, I_2, \dots, I_n\})$

#MOTION DETECTION

```
import cv2

cap = cv2.VideoCapture(0)

ret, frame1 = cap.read()
ret, frame2 = cap.read()

while True:

    diff = cv2.absdiff(frame1, frame2)
    gray = cv2.cvtColor(diff, cv2.COLOR_BGR2GRAY)

    blur = cv2.GaussianBlur(gray, (5,5), 0)

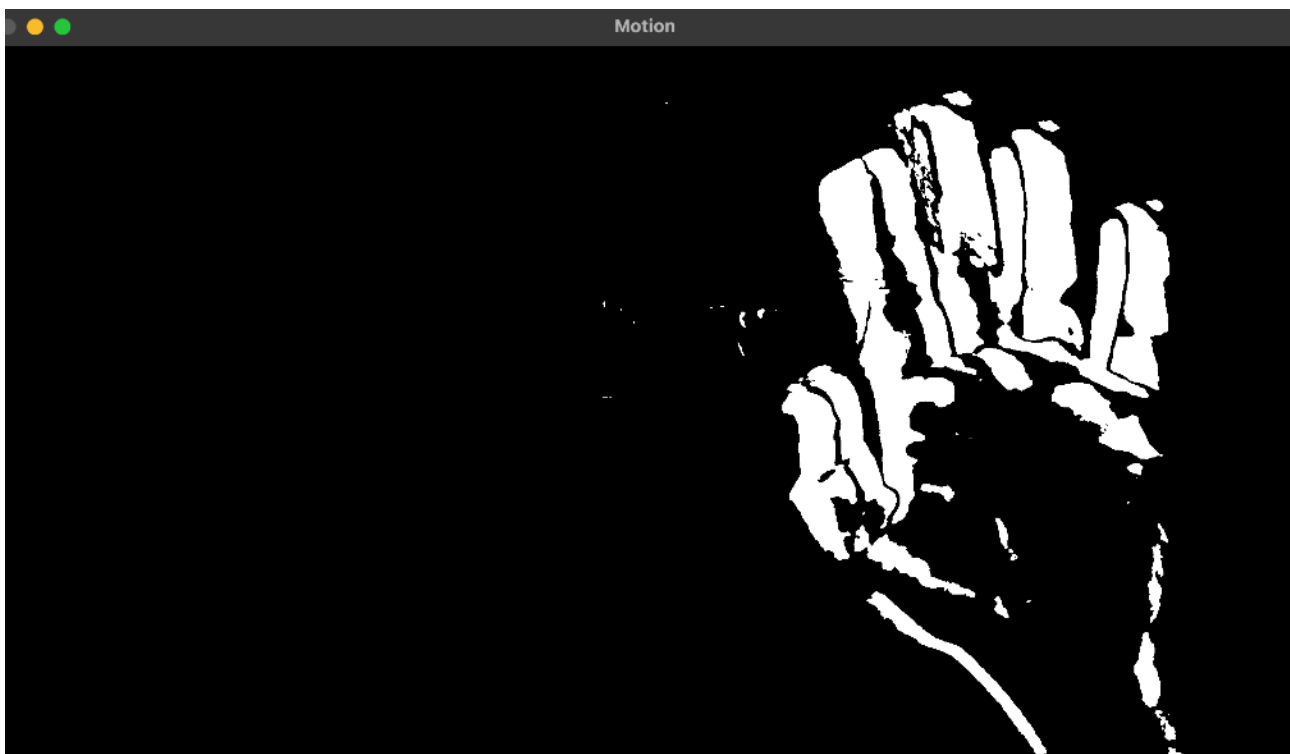
    _, thresh = cv2.threshold(blur, 20, 255, cv2.THRESH_BINARY)

    cv2.imshow("Motion", thresh)

    frame1 = frame2
    ret, frame2 = cap.read()

    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```



Capture two frames

Compute frame difference

Convert to grayscale

Apply Gaussian blur

Apply threshold

Display motion areas

Repeat continuously